Linear-Time Sorting

Peerapon S.

CPE 231 Algorithms

Topics

Counting sort

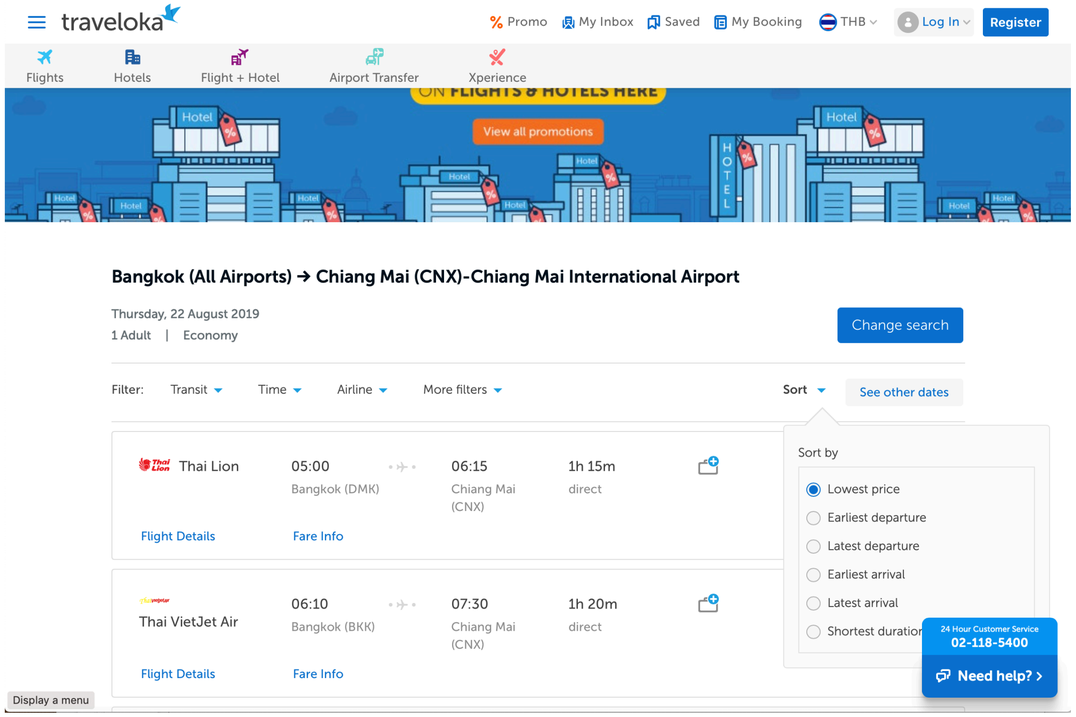
Radix sort

Sorting Problem

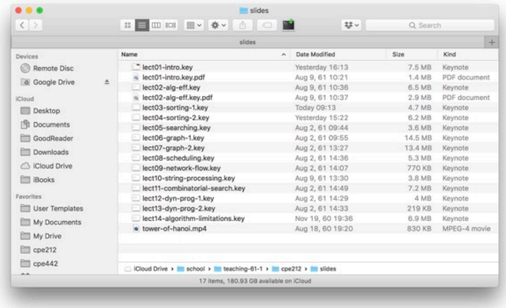
Most fundamental problem in studying of algorithms.

- Lots of applications inherently need to sort information.
- Often used as a key subroutine in other algorithms

Some problems efficiently solved when inputs are presorted, e.g., element uniqueness, mode, binary search.







Stable vs. Unstable sorting

What is the requirement difference in sorting two lists below ?

32
37
57
40
50
36
35
42
45
43
56
38
56
36
38
37
53
52
33
43
32
43
38
33
57
57
41
39
35
31
52
40
41

Akkharaphon	32
Arnukool	37
Chanchayut	57
Jarupa	40
Jeeravit	50
Jiranan	36
Kanjanadit	35
Kanyawee	42
Karnrawee	45
Kaung-Htet	43
Kriattiyod	56
Marida	38
Naphon	56
Natvara	36
Nay-Naing	38
Nonlapun	37
Parin	53
Pasudm	52
Patisatorn	33
Penpitchaya	43
Phatcharaphong	32
Piyaporn	43
Qiufan	38
Raweenipha	33
Sadab	57
Saowaphak	57
Sarun	41
Tharuesida	39
Tinnapat	35
Traithep	31
Treenoppanut	52
Vasin	40
Vibhu	41

Summary of Comparison-based Sorting

Algorithm	Worst-case Run-time	Stable	When to use
Insertion sort	$\Theta(n^2)$	**Yes**	Small list < few k Almost-sorted list
Selection sort	$\Theta(n^2)$	**Yes**	Small list < few k Slow memory access
Bubble sort	$\Theta(n^2)$	**Yes**	Rarely used
Heap sort	$\Theta(n \lg n)$	**No**	Large list with limited memory space
Merge sort	$\Theta(n \lg n)$	**Yes**	Large list w/o memory-constrained
Quick sort	$\Theta(n^2)$ $\Theta(n \lg n)$ average	**No**	Large list w/o too many duplicates

Beating the comparison-based sorting

Any ***comparison-based sorting*** has lowerbound $\Omega(n \lg n)$.

Can we do better?

- Yes ! by ***not*** comparing two keys directly.
- Applied to a list with **known finite set of values** (e.g., student scores, ages, rating scores, etc.)

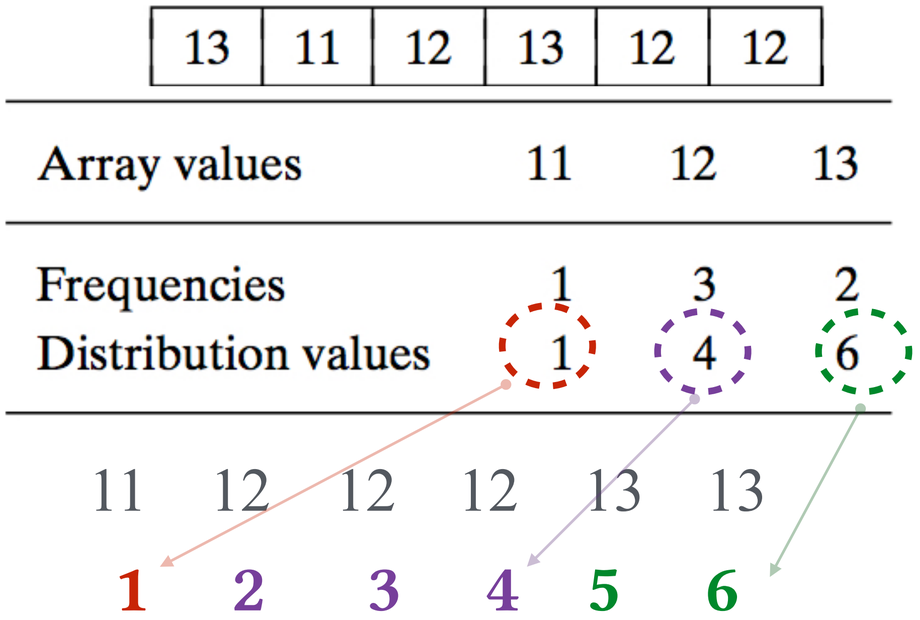
Large commercial platforms sort and bucket tens of millions of active users by discrete demographic attributes (like age, rating scores) to serve targeted recommendations or cohort reports.

Achieving better sorting performance than $\Theta(n \lg n)$ is definitely valuable.

ID	Age	Rating Score (1-10)	Gender	Education Level	Income Bracket
1001	29	9	Female	Bachelor's	Mid-Income
1002	42	9	Male	Master's	High-Income
1003	21	6	Non-Binary	Some College	Low-Income
1004	55	8	Male	Doctorate	High-Income
1005	34	9	Female	Bachelor's	Mid-Income
1006	38	4	Male	High School	Low-Income
1007	47	10	Female	Master's	High-Income
1008	26	7	Female	Bachelor's	Low-Income
1009	61	8	Male	Associate's	Mid-Income
1010	31	9	Non-Binary	Master's	Mid-Income

(Distribution) Counting Sort

Initially, count the frequencies to determine the (cumulative) distribution.



Distribution value determines indices of that number in sorted output.
Ex: 11 at 1
12 at 4, 3, 2
13 at 6, 5

Need two extra arrays — (cumulative) Distribution $D[]$ and Sorted output $S[]$.

For each value in input $A[]$ from right to left

- Place the value at the index equal to its distribution in $S[]$ and decrease the distribution in $D[]$ by one.
- Process the list from right to left (why?)

A[] =

13	11	12	13	12	12

D[]

Index	11	12	13
1st iteration	1	4	6
2nd iteration			
⋮			
6th iteration			

S[]

1	2	3	4	5	6
			12		

Algorithm CountingSort(A, l, u)

1: Input: Array $A[1 \ldots n]$ of integers between l and u $(l \leq u)$ $m = u - l + 1$ unique values
2: Output: Array $S[1 \ldots n]$ of $A[]$ sorted in non-decreasing order.
3:
4: **for** $j = 1$ to $u - l + 1$ **do** $D[j] \leftarrow 0$
5: **for** $i = 1$ to n **do** ▷ Compute the frequencies
6: $\quad v \leftarrow A[i] - l + 1$
7: $\quad D[v] \leftarrow D[v] + 1$
8: **for** $j = 2$ to $u - l + 1$ **do** ▷ Compute the cumulative distribution
9: $\quad D[j] \leftarrow D[j - 1] + D[j]$
10: **for** $i = n$ downto 1 **do** ▷ Put items into sorted output
11: $\quad j \leftarrow A[i] - l + 1$
12: $\quad S[D[j]] \leftarrow A[i]$
13: $\quad D[j] \leftarrow D[j] - 1$
14: **return** S

Sorting Fixed-Length Strings

How to sort 6-character airline confirmation code ?

How to get unique product serial numbers in customer baskets from RFID scanning checkout?

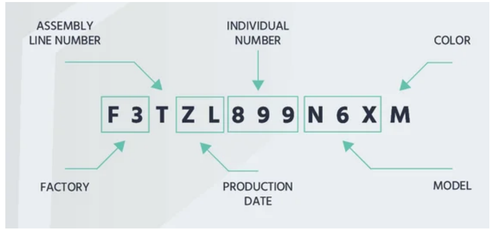
```
ISSUING AIRLINE                    : THAI AIRWAYS INTERNATIONAL
TICKET NUMBER                      : ETKT 217
BOOKING REF : AMADEUS: 7QXXVF, AIRLINE: TG/7QXXVF

FROM /TO          FLIGHT  CL  DATE    DEP      FARE BASIS     NVB    NVA    BAG   ST

SEOUL INCHEON     TG 657  L   25MAY   2120     LRL21KR                15JUN  30K   OK
INTERNATIONAL
BANGKOK              SEAT: 49A     ARRIVAL TIME: 0110    ARRIVAL DATE: 26MAY
SUVARNABHUMI
INTL
```







Duplicate S/N of items from bar-code scanning
How to get only unique items ?

11111-55555-31003
11111-55555-21789
11111-55555-45732
11111-55555-45732
11111-55555-21789
11111-55555-31003
..

Radix Sort

Use counting sort as a subroutine.

Apply counting sort with rightmost character as the sort key, then the leftmost as the sort key.

F6, E5, R6, X6, X2, T5, F2, T3



X2, F2, T3, E5, T5, F6, R6, X6



E5, F2, F6, R6, T3, T5, X2, X6

Input: XI7FS6; PL4ZQ2; JI8FR9; XL8FQ6; PY2ZR5; KV7WS9; JL2ZV3; KI4WR2

Six sort keys where each sort key is a 6-digit number and each digit is the range 0 - 35.

i	Resulting order
1	⟨PL4ZQ2, KI4WR2, JL2ZV3, PY2ZR5, XI7FS6, XL8FQ6, JI8FR9, KV7WS9⟩
2	⟨PL4ZQ2, XL8FQ6, KI4WR2, PY2ZR5, JI8FR9, XI7FS6, KV7WS9, JL2ZV3⟩

Self Test : Linear-Time Sorting



https://forms.office.com/r/0SzgiZRWrc?origin=lprLink